

Computing Energy Optimisation for D-ITET

Research Group

Michael Young

Evgeny Kartashov

Tobias Gut

Malin Koyupinar

Supervisor

Benjamin Weber

Akor Innocent

Examiner

Prof. Dr. Lana Josipović

Prof. Dr. Gabriela Hug

Report Date

December 16, 2025

Department of Information Technology and Electrical Engineering

ETH Zurich

Motivation

Computing environments often grow organically over many years. This leads to situations where machines are added for specific tasks but not fully integrated afterwards. In many cases systems remain running even though the services they originally hosted are no longer required. They continue to draw power and generate heat without providing meaningful functionality. Several IT teams are aware of these issues and have attempted different solutions, such as consolidating workloads through virtual machines, using shared systems instead of maintaining multiple individual servers and installing automatic relays for desks and equipment. These attempts demonstrate that the problem is not a lack of technical solutions. Instead, the main challenges are working hours, financial constraints and the difficulty of evaluating which optimisation measures are worth implementing. A systematic overview of the current situation is therefore necessary to support informed decision-making.

Acknowledgements

First and foremost, we would like to express our sincere gratitude to the P&S initiative “Let’s Make ITET Green” for providing the framework and motivation for this project. We are especially thankful to Benjamin Weber for his consistent support throughout the work. His commitment to meeting with us regularly, his clear guidance during discussions, and the time he dedicated to answering our emails were extremely helpful. His suggestions helped us refine our approach, understand the broader context of the project, and move in a clear and well-structured direction.

Furthermore, we would like to extend our thanks to the ITET IT team for their essential support, specifically Thomas Berchtold and Niklaus Kappeler. Their expertise and support regarding the technical infrastructure were invaluable to the successful realisation of our work.

Contents

1	Introduction	1
2	Analysis of available data	3
2.1	Zabbix Monitoring Data	3
2.1.1	Overview of Zabbix at D-ITET	3
2.1.2	Case Study: Host ee-tik-hv002.ethz.ch	3
2.1.3	Power readings in Zabbix	5
3	Automated power measurement strategy	6
3.1	Qualitative Strategy (Script Analysis)	6
3.1.1	Facility Level (PDU/Smart Plug)	6
3.1.2	Server Level (BMC - Redfish/IPMI)	6
3.1.3	System Level (OS - RAPL/Powercap)	7
3.1.4	Component Level (GPU)	7
3.2	Discussion: Accuracy Analysis	7
3.2.1	The “Blind Spot” Phenomenon	7
3.2.2	Impact of Load on Measurement Accuracy	7
4	Implementation of Power Telemetry Framework	9
4.1	Design objectives and operational constraints	9
4.2	Architecture and separation of concerns	10
4.3	Repository organisation and reproducibility	10
4.4	Deployment workflow and idempotent installation	12
4.4.1	Capability detection and tri-state configuration semantics	12
4.4.2	CPU package power via RAPL	12
4.4.3	RAM power via RAPL DRAM domain	12
4.4.4	Fallback: AMD CPU power via <code>hwmon</code>	13
4.4.5	GPU power: vendor tools and sysfs fallbacks	13
4.4.6	Platform power via Redfish/IPMI	13
4.5	Measurement scripts: contracts, algorithms, and failure semantics	13
4.5.1	RAPL power computation from energy deltas	13
4.5.2	GPU power extraction	14
4.5.3	Performance constraints and why discovery is moved to deployment time	14
4.6	Zabbix integration: UserParameters and GPU Low-Level Discovery	14
4.6.1	Agent configuration generation	14
4.6.2	GPU discovery and dynamic item creation	15

4.6.3	Rationale for manual server-side configuration in the PoC	15
4.7	Security and operational considerations	15
4.8	Limitations and future work	16
5	Component Efficiency and Optimisation	17
5.1	PSU	18
5.1.1	Replacement options PSU	19
5.2	CPU	19
5.2.1	Replacement options CPU	20
6	Conclusion	21
	Bibliography	23

Chapter 1

Introduction

Our research group — Michael Young, Evgeny Kartashov, Tobias Gut, and Malin Koyupinar — is investigating the topic of Computing Energy Optimisation for D-ITET. The project focuses on understanding how computing resources within the department are currently used, how much power they consume, and which strategies could increase their overall efficiency. To achieve this, we structured our work into a coherent narrative that moves from a high-level analysis of available data to technical measurement strategies, and finally to concrete hardware assessments.

First, we examined the available data to understand the current landscape. We accessed two primary sources: the Zabbix monitoring system, which provides historical information on machine activity and utilisation, and the local `invcxseth` inventory platform, which lists machine specifications and installed components. Upon analysing these sources, we discovered a significant disconnect. While performance metrics such as CPU usage and disk space are abundant, direct power measurements are clearly missing for the vast majority of systems. This absence of energy data prevents informed decision-making, as it is impossible to effectively optimise what cannot be measured.

Second, having identified this lack of data as the primary bottleneck, we investigated how to measure power automatically and reliably across a heterogeneous infrastructure. We evaluated different telemetry sources, including IPMI, Redfish, and RAPL, and other vendor-specific solutions to determine how they could be deployed to fill the data gap. A crucial part of this investigation was identifying the limitations of software-based sensors, specifically the “blind spot phenomenon,” where internal sensors fail to capture the base load of fans and power supply inefficiencies. This analysis lays the groundwork for a unified measurement script capable of generating a true energy baseline.

Third, we translated this measurement strategy into an operational proof-of-concept that can be rolled out at department scale. Concretely, we implemented an Ansible-managed Zabbix power telemetry framework that performs capability discovery at deployment time (e.g., detecting available RAPL domains, vendor GPU tooling such as `nvidia-smi`/`rocm-smi`, and optional platform interfaces) and then deploys a minimal set of host-local measurement scripts plus a Zabbix agent configuration fragment. The framework is designed to be compatibility-first and non-intrusive: it avoids persistent background services beyond the existing Zabbix agent, keeps runtime polling lightweight (simple `sysfs` reads or a single CLI call), and fails cleanly when an interface is unsupported. For heterogeneous accelerator nodes, GPU metrics are exposed using Zabbix Low-Level Discovery so items adapt automatically to the number of devices present. This proof-of-concept demonstrates a practical path from “missing power data” to reliable, time-series energy

telemetry without requiring invasive software installation.

Fourth, using the hardware specifications from the inventory, we looked at the power efficiency of the most popular components currently in use. Even in the absence of live power logs, the theoretical efficiency of Power Supply Units (PSUs) and CPUs offer strong indicators of energy waste. By comparing legacy components against modern standards, we proposed specific replacement options. This analysis establishes that there is a clear potential for improvement, demonstrating that targeted hardware updates can yield significant energy savings even before complex software optimisations are applied.

The following chapters detail these four steps, providing a roadmap from the current state of missing data to a future state of measurable, optimised efficiency.

Chapter 2

Analysis of available data

In this chapter, we analyse the existing datasets available to us, identify missing metrics, and outline our validation strategies.

2.1 Zabbix Monitoring Data

Zabbix is an open-source monitoring and observability system designed to supervise IT and OT infrastructures [1]. It collects a wide range of performance metrics such as CPU utilisation, memory usage, load average, and disk usage and offers long-term historical graphs for each monitored host. For the analysis in this chapter, all Zabbix plots were exported for the same time period (from 29.05.2024 to 29.11.2024). At D-ITET, the Zabbix instance provides an overview of all monitored systems and displays hosts, tags, and metrics in a structured way.

2.1.1 Overview of Zabbix at D-ITET

The host list shows a heterogeneous environment consisting of physical compute nodes (`ee-tik-cnXXX`), virtual machines (`ee-tik-vmXXX`), hypervisors (`ee-tik-hvXXX`), and workstation systems. Despite this breadth of operational data, the monitoring setup contains significant gaps with respect to energy analysis. While Zabbix provides detailed performance metrics for almost all hosts, power consumption values are only available for a very small number of systems. This strongly limits the system’s usefulness for energy optimisation and motivates the additional measurement strategy discussed in the following chapter.

2.1.2 Case Study: Host `ee-tik-hv002.ethz.ch`

For detailed analysis, we selected the host `ee-tik-hv002.ethz.ch` as a representative example. This machine is one of the most comprehensively monitored systems in the D-ITET Zabbix environment.

CPU Utilisation

For CPU utilisation we selected the long-range graph covering the full observation window (see Figure 2.1). Over this period, the system spends extended phases in the 0–3% range, with only occasional short spikes. This behaviour is representative of many servers in the environment and has direct implications for energy optimisation. Even though the CPU

is almost idle most of the time, this does *not* imply that the server’s overall power draw is low. As discussed later in the context of the “blind spot” phenomenon, CPU counters only capture the active silicon power and do not reflect the constant overhead from fans, PSUs and baseboard components. The dominance of low utilisation therefore highlights why understanding and measuring idle and low-load power is crucial.

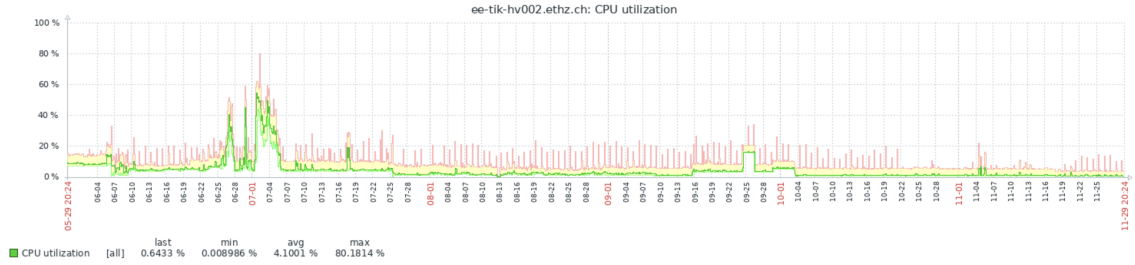


Figure 2.1: CPU utilisation of `ee-tik-hv002.ethz.ch`.

Load Average

The load-average graph provides three complementary time scales—1 min, 5 min, and 15 min. Throughout the six-month observation window, all three curves remain well below the system’s logical CPU count. Throughout the observation window, the load-average curve closely mirrors the CPU-utilisation profile. This behaviour indicates that the system operates predominantly in a low-load regime, with only a small number of active processes and minimal waiting for CPU resources. Consequently, short CPU activity bursts appear in both metrics, whereas prolonged idle periods result in consistently low values. The similarity thus indicates a lack of scheduling contention, not a redundancy of the measures. Figure 2.2 illustrates the 1 min load-average time scale as a representative example.

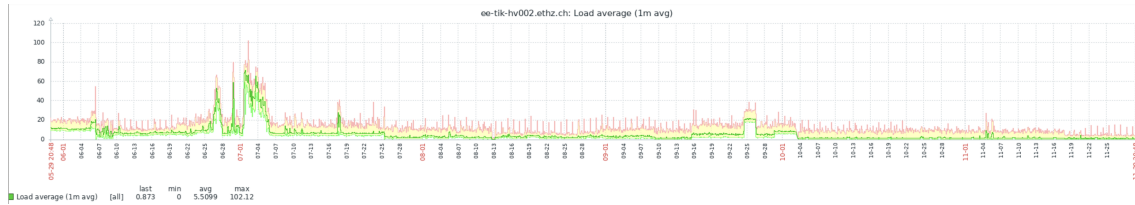


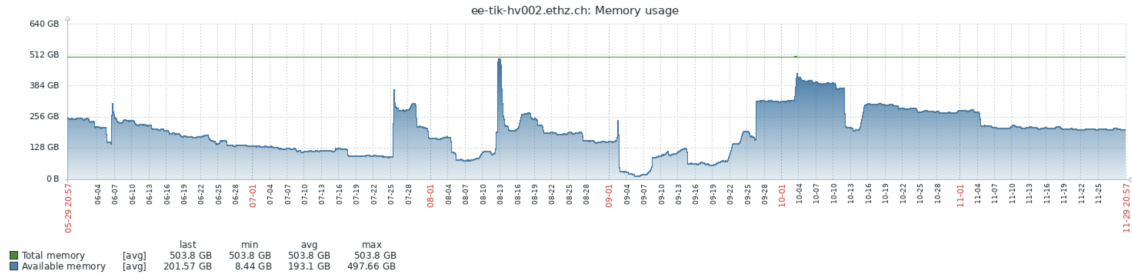
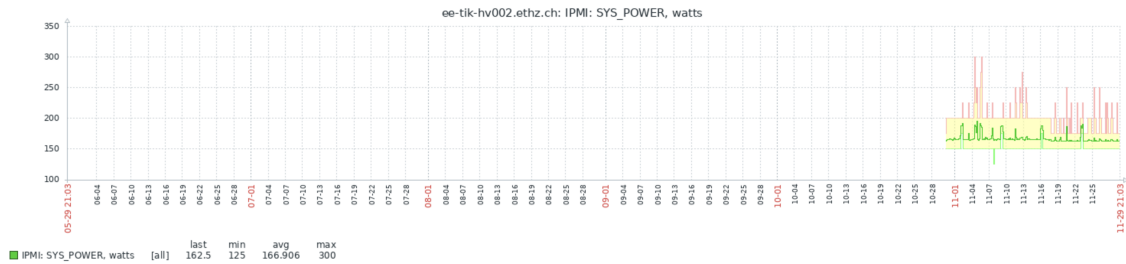
Figure 2.2: Load average (1 min) of `ee-tik-hv002.ethz.ch`.

Memory Usage

The memory usage graph reveals both gradual trends and several short-lived fluctuations (see Figure 2.3). The total memory capacity of approximately 512 GB stays constant, while the available-memory curve shows a steady downward drift.

IPMI System Power (Watts)

The IPMI system power measurement provides a direct estimate of the machine’s electrical load (see Figure 2.4). The available data begins only in early November, leaving a large gap in the historical record, yet within the visible interval the system remained mostly steady between 150–180 W.

Figure 2.3: Memory usage of `ee-tik-hv002.ethz.ch`.Figure 2.4: IPMI system power of `ee-tik-hv002.ethz.ch`.

2.1.3 Power readings in Zabbix

While the case study of `ee-tik-hv002` presented in the previous section includes a clear time-series graph for electrical power draw (Figure 2.4), our investigation reveals that this is an exception rather than the norm. A systematic review of the remaining hosts in the Zabbix inventory shows that the vast majority of systems lack any form of direct power telemetry.

The `ee-tik-hv002` server is one of the few instances where the IPMI sensors are correctly mapped and logging data to the monitoring system. For the remainder of the infrastructure the “watts” or “energy” metrics are either missing entirely from the Zabbix templates or remain unpopulated. This absence of data presents a fundamental barrier to optimisation; without a baseline of actual power consumption, it is impossible to identify energy hotspots or accurately quantify the impact of any efficiency improvements.

This key finding—that the current monitoring infrastructure excels at tracking performance (CPU, RAM, Disk) but effectively ignores energy—defines the primary technical challenge of this project. Consequently, our focus must shift from analysing existing logs to establishing a method for extracting power readings directly from the hardware. The strategy to bridge this gap and automate the collection of these missing metrics is detailed in the following chapter.

Chapter 3

Automated power measurement strategy

To accurately profile the energy consumption of the D-ITET infrastructure, we require a multi-layered measurement strategy. Essentially, we aim to deploy various scripts and tools to measure power at different granularities across our servers. The following sections outline our qualitative strategy for script selection, the plan for deployment, and a discussion on measurement accuracy.

3.1 Qualitative Strategy (Script Analysis)

We have identified four distinct levels of power measurement, ranging from the facility level down to individual components.

3.1.1 Facility Level (PDU/Smart Plug)

This level represents the widest scope of measurement, capturing the Total Wall-Draw.

Methodology: Utilises SNMP queries to interface with rack-level Power Distribution Units (PDUs) (e.g., APC, Eaton).

Utility: It serves as the “ground truth” for total energy consumption, as it accounts for Power Supply Unit (PSU) inefficiencies and cooling overhead. It is best suited for capacity planning and total cost of ownership calculations.[2]

3.1.2 Server Level (BMC - Redfish/IPMI)

This level offers the optimal balance for server management, measuring Total Hardware Power.

Methodology: Relies on Out-of-Band management via the Baseboard Management Controller (BMC). It utilises the modern Redfish REST API for automated JSON retrieval or the legacy IPMI standard for broad compatibility across rack servers (Dell, HPE, Lenovo).

Utility: This method captures power data independently of the Operating System, eliminating OS overhead while ensuring total hardware draw is recorded. It is the preferred standard for server-level profiling. [3]

3.1.3 System Level (OS - RAPL/Powercap)

This level focuses on high granularity within the compute node, specifically CPU and DRAM Power.

Methodology: Utilises In-Band, OS-level tools to read Intel/AMD Running Average Power Limit (RAPL) counters. Data is acquired via `turbostat` or by calculating the delta of raw energy counters in `sysfs` over time:

$$P_{\text{watts}} = \frac{E_{t_2} - E_{t_1}}{\Delta t}$$

Utility: Ideal for profiling specific software workloads on personal computers or systems lacking BMCs. However, it notably excludes overhead from fans, PSUs, and peripheral components. Accuracy is high “for loads above idle” [4]

3.1.4 Component Level (GPU)

This level provides isolation for high-performance accelerators.

Methodology: Queries vendor-specific drivers such as `nvidia-smi` for NVIDIA GPUs or `rocm-smi` for AMD GPUs to extract instantaneous power draw. [5] [6]

Utility: Essential for AI/ML and rendering workloads where the accelerator consumes the majority of the energy budget, allowing researchers to decouple accelerator efficiency from the host system.

3.2 Discussion: Accuracy Analysis

When managing a fleet of approximately 250 compute nodes, relying solely on System (RAPL) or Component (GPU) level sensors introduces a variable error margin relative to the true facility energy cost. This deviation is not constant; it is inversely proportional to the system load.

3.2.1 The “Blind Spot” Phenomenon

System-level counters (RAPL) and Component drivers (NVIDIA-SMI) measure active silicon power (CPU, DRAM, GPU) but remain blind to peripheral consumers such as fans, motherboard chipsets, and Power Supply Unit (PSU) conversion losses.

As established by the analysis of Hackenberg et al. (2015), this creates a significant measurement gap. While RAPL registers are highly precise for the specific domains they monitor (often deviating $< 2\%$ for the CPU package), they exclude the constant “base load” of unmeasured components. Therefore, they cannot serve as a direct proxy for total system power without calibration.

3.2.2 Impact of Load on Measurement Accuracy

The relationship between sensor readings and total wall-draw changes across three distinct utilisation states:

- Idle State (High Relative Error):** At 0% utilisation, the CPU and GPU enter deep C-states, reducing their power consumption to near zero (e.g., 20W). However, the unmeasured “base load” (fans, BMC, PSU leakage) remains constant (e.g., 50W). In this scenario, RAPL might capture only $\approx 30\%$ of the actual energy being drawn from the PDU.
- 50% Partial Load (Transition Zone):** As workload increases, the active silicon power rises significantly. The ratio of “measured” power (CPU/GPU) to “unmeasured” power (chassis overhead) improves. While the sensor readings begin to correlate more strongly with PDU trends, the exclusion of fan ramp-up curves can still lead to non-linear inaccuracies.
- 100% Full Load (Optimal Correlation):** At maximum utilisation, the processors draw their peak TDP (e.g., 250W+), rendering the fixed chassis overhead a much smaller fraction of the total. Consequently, the System and Component level measurements align most closely with the total facility draw percentage-wise.

Chapter 4

Implementation of Power Telemetry Framework

4.1 Design objectives and operational constraints

The primary objective of the implementation was to close the empirical gap identified in Chapter 2: although Zabbix provides extensive utilisation and performance telemetry, power readings are absent for most hosts in the department environment. The design target was therefore not a laboratory-grade measurement appliance, but an operationally deployable framework capable of producing sufficiently reliable *time-series power telemetry* across a heterogeneous fleet.

The department infrastructure exhibits strong diversity in hardware generations, vendors, and host roles (compute nodes, workstations, and specialised servers). In such settings, traditional approaches that rely on installing additional measurement daemons or vendor SDKs on every target create unacceptable operational risk: unsupervised package installation may collide with local configurations, violate change-management constraints, or require administrative effort that does not scale. Consequently, the implementation was guided by four constraints:

1. **Compatibility-first and minimal dependencies:** runtime measurements should rely on interfaces that are already present on the host (e.g., `sysfs` power counters provided by the Kernel or vendor-provided CLIs when installed).
2. **Non-intrusiveness:** the framework should avoid introducing persistent background services beyond the already deployed Zabbix agent, and it must be safe to deploy on systems with unknown or custom configuration.
3. **Separation of discovery and polling:** expensive enumeration (hardware detection and capability mapping) should occur at *deployment time*, not at every measurement poll.
4. **Idiomatic integration with existing infrastructure:** deployment must be managed via Ansible, and data collection must be exposed to Zabbix¹ in the standard way (agent `UserParameter` keys and Low-Level Discovery where needed).

¹Such decision was taken after a short feasibility analysis that was aimed to identify infrastructure software components that one would assume as backbone of the system and build our own software stack on their base.

These constraints deliberately bias the framework towards a robust proof-of-concept (PoC) suitable for department-scale roll-out, rather than towards maximising coverage through invasive and high maintenance effort instrumentation.

4.2 Architecture and separation of concerns

The framework follows a control-plane / data-plane decomposition:

Control plane (Ansible). Ansible performs *capability discovery* at deployment time (e.g., determining whether RAPL counters exist, whether `nvidia-smi` or `rocm-smi` is available, and whether Redfish credentials are configured). Based on the discovery result, Ansible renders and deploys a small set of host-local measurement scripts and a Zabbix agent configuration fragment.

Data plane (Zabbix agent + scripts). The Zabbix agent executes measurement scripts on demand when polled by the Zabbix server. Each script returns an instantaneous power reading in watts, or fails cleanly if the interface is not supported. This execution model ensures that no continuous sampling daemon is required on the host.

Aggregation (Zabbix server). The Zabbix server polls the defined `UserParameter` item keys at configured intervals and stores them as time series. GPU heterogeneity is handled using Zabbix Low-Level Discovery (LLD), allowing GPU indices to be discovered dynamically.

This separation renders the runtime system cost-effective: measurement-time execution is confined to reading one or a small number of kernel-exposed counters or invoking a single vendor CLI command. Hardware detection and configuration synthesis are shifted to Ansible deployment time, where performance is less critical.

4.3 Repository organisation and reproducibility

To support reproducibility and maintainability, the implementation is structured as a dedicated Ansible role. The repository is organised around a single deployment playbook and a single monitoring role:

- **Inventory and playbook level:** an Ansible inventory defines target groups and connectivity methods; the playbook applies the power monitoring role to the target groups.
- **Role level:** `roles/power_monitoring/` contains:
 - `tasks/`: capability detection tasks and deployment orchestration.
 - `templates/`: Jinja2 templates for scripts and the Zabbix agent configuration fragment.
 - `vars/`: default role configuration (feature flags, recommended polling interval).

The use of Jinja2 templates is not cosmetic: templates allow Ansible to parameterise measurement scripts and agent configuration based on discovered interfaces (e.g., number of CPU packages, availability of DRAM RAPL domains, or vendor-specific GPU tooling). This keeps runtime scripts simple and avoids repeated expensive detection during polling.

```

.
├── ansible.cfg
├── group_vars
│   └── generic_server.yml
├── host_vars
│   └── generic-server.yml
├── inventory
│   └── hosts.ini
├── playbooks
│   └── deploy-power-monitoring.yml
├── README.md
├── roles
│   └── power_monitoring
│       ├── handlers
│       │   └── main.yml
│       ├── tasks
│       │   ├── deploy_agent_config.yml
│       │   ├── deploy_scripts.yml
│       │   ├── detect_gpu_amd.yml
│       │   ├── detect_gpu_nvidia.yml
│       │   ├── detect_ram_rapl.yml
│       │   ├── detect_rapl.yml
│       │   ├── detect_redfish.yml
│       │   └── main.yml
│       ├── templates
│       │   ├── discover_gpu_sensors.sh.j2
│       │   ├── power.conf.j2
│       │   ├── read_cpu_amd_hwmon.sh.j2
│       │   ├── read_cpu_rapl_power.sh.j2
│       │   ├── read_gpu_amd_power.sh.j2
│       │   ├── read_gpu_nvidia_power.sh.j2
│       │   ├── read_ram_rapl_power.sh.j2
│       │   └── read_redfish_power.sh.j2
│       └── vars
│           └── main.yml

```

Figure 4.1: Reference Project Structure

4.4 Deployment workflow and idempotent installation

Deployment is executed from an administrative workstation using Ansible, targeting hosts over local connection or over Secure Shell (SSH) depending on host type. The playbook applies the monitoring role with privilege escalation, and the role performs the following steps:

1. **Detect capabilities:** CPU package power interfaces, RAM power interfaces, GPU interfaces, and optional platform power via Redfish/IPMI.
2. **Render scripts:** generate host-local scripts under a dedicated directory (e.g., `/usr/local/share/zabbix_power_monitoring/`).
3. **Render agent configuration:** generate a Zabbix agent configuration fragment that maps each metric to a stable key via `UserParameter`.
4. **Activate configuration:** restart or reload the Zabbix agent to apply the configuration change.

A critical property is *idempotence*: repeated playbook runs should converge on the same state without accumulating duplicate configuration or diverging behaviour. This is essential for safe operation in environments where hosts may be reimaged, upgraded, or repurposed.

4.4.1 Capability detection and tri-state configuration semantics

In heterogeneous fleets, strict assumptions cause deployment failure. Conversely, fully best-effort approaches may produce silent gaps that are difficult to debug. To balance these concerns, each component class (CPU, RAM, GPU, Redfish) supports a tri-state enable flag:

auto: best-effort; detect what is available and deploy only supported metrics.

true: require the capability; fail deployment if the interface is missing.

false: disable the component entirely.

This pattern allows the same codebase to support both exploratory PoC rollouts (**auto**) and controlled experiments where power telemetry is a hard requirement (**true**).

4.4.2 CPU package power via RAPL

For CPU package power, the preferred mechanism is RAPL (Running Average Power Limit) energy counters exposed through the Linux powercap subsystem. At deployment time, the role checks for the existence of RAPL powercap directories and enumerates package-level `energy_uj` files. Systems with multiple sockets yield multiple package counters, which are then mapped to distinct Zabbix keys (e.g., `power.cpu.package0`, `power.cpu.package1`).

4.4.3 RAM power via RAPL DRAM domain

If the platform exposes a DRAM RAPL domain, the role similarly enumerates the corresponding `energy_uj` files and creates per-domain Zabbix keys (e.g., `power.ram.package0`). If no DRAM domain exists, RAM power measurement is skipped rather than emulated.

4.4.4 Fallback: AMD CPU power via hwmon

Some AMD systems do not expose RAPL domains in a way that is accessible or supported in the deployment context. For such hosts, the framework includes a fallback detector that searches for AMD CPU power reporting via `hwmon` (e.g., `fam15h_power`, `zenpower`, or platform-specific sensor implementations). This fallback is explicitly treated as a secondary path to avoid conflating heterogeneous sensor semantics.

4.4.5 GPU power: vendor tools and sysfs fallbacks

GPU power telemetry is obtained through vendor tooling when available:

- **NVIDIA:** `nvidia-smi` provides per-GPU instantaneous power draw. The role deploys a per-index query script and exposes a Zabbix key family `power.gpu[*]`.
- **AMD:** `rocm-smi` is the preferred user-facing interface when installed; if unavailable or unsupported, `sysfs/hwmon` power input counters can serve as a fallback on some systems.

Because GPU inventory can change (and because multi-GPU systems are common for accelerator workloads), the framework also exposes a GPU Low-Level Discovery mechanism, allowing Zabbix to dynamically create per-GPU items.

4.4.6 Platform power via Redfish/IPMI

Platform-level PSU power (closest to total server draw) is architecturally supported through a Redfish/IPMI script hook. However, due to the variability of BMC endpoints, authentication models, and local network policy, Redfish is implemented as an explicit opt-in path that requires host-specific configuration. In the current PoC, the Redfish script is treated as a placeholder interface for subsequent iteration.

4.5 Measurement scripts: contracts, algorithms, and failure semantics

All measurement scripts follow a strict contract:

- On success, print exactly one numeric value in watts and exit with status code 0.*
- On failure, print nothing and exit with a non-zero code.*

This contract is not merely stylistic. It avoids ambiguous states in monitoring: a numeric 0 is a valid measurement in some contexts, whereas “no output + non-zero exit” clearly signals “metric not supported” or “measurement failed”. Zabbix can then classify the item appropriately without preprocessing heuristics.

4.5.1 RAPL power computation from energy deltas

RAPL counters typically expose cumulative energy in microjoules. Instantaneous power is estimated by measuring the delta between two reads separated by a short sampling interval:

$$P = \frac{\Delta E}{\Delta t} \quad \text{with} \quad \Delta E = E(t_2) - E(t_1).$$

With ΔE in microjoules, the implementation converts to joules and reports watts accordingly. The sampling interval Δt is chosen small enough to provide resolution yet large enough to reduce quantisation noise, and it is implemented using a fixed sleep in the script.

An engineering complication is counter wraparound: if the energy counter overflows between reads, ΔE becomes negative. Production-grade implementations therefore include a correction term using the maximum counter range where available. The PoC design anticipates this behaviour and structures scripts such that wraparound handling can be added without altering the Zabbix interface contract.

4.5.2 GPU power extraction

For GPUs, the measurement model is vendor-defined:

- NVIDIA tooling reports power draw directly in watts (or as a unitless numeric that can be interpreted as watts when queried in `nounits` mode).
- AMD tooling may return N/A or “not supported” on some systems; scripts must treat this as a hard failure rather than returning an arbitrary number.

These cases illustrate a general principle: *do not synthesise data*. If a system does not support a reliable GPU power reading, the correct behaviour is to expose “not supported” to Zabbix and rely on alternative measurement layers (e.g., BMC-level telemetry or facility-level metering) for that host class.

4.5.3 Performance constraints and why discovery is moved to deployment time

Zabbix deployments often poll items at multi-second intervals. Therefore, per-poll scripts must be cheap and deterministic. The framework enforces this by:

1. restricting expensive enumeration (e.g., scanning PCIe inventory) to deployment time; and
2. keeping runtime scripts limited to small sysfs reads or a single vendor CLI call.

This design reduces both measurement overhead and operational unpredictability.

4.6 Zabbix integration: UserParameters and GPU Low-Level Discovery

Zabbix integration is implemented through agent-side `UserParameter` keys. For CPU and RAM, keys are statically generated based on the number of detected packages or domains. For GPUs, keys are implemented as an indexed family to support multiple devices per host.

4.6.1 Agent configuration generation

The role generates an agent configuration fragment (e.g., `power.conf`) that binds each measurement to a Zabbix key. Conceptually, the configuration follows the pattern:

```

UserParameter=power.cpu.package0, /path/read_cpu_rapl_power.sh 0
UserParameter=power.ram.package0, /path/read_ram_rapl_power.sh 0
UserParameter=power.gpu[*], /path/read_gpu_power.sh "$1"
UserParameter=power.gpu.discovery,/path/discover_gpu_sensors.sh

```

The key design goal is stability: a given host produces the same key names across redeployments, while still adapting to different hardware configurations.

4.6.2 GPU discovery and dynamic item creation

GPU Low-Level Discovery (LLD) is used to keep the Zabbix configuration consistent with the actual GPU inventory. The discovery script returns a JSON structure describing available GPU indices and (optionally) vendor metadata. Zabbix then creates item prototypes (e.g., `power.gpu[0]`, `power.gpu[1]`) based on that discovery.

This approach avoids hard-coding the number of GPUs in templates and supports accelerator servers where GPU configurations vary over time.

4.6.3 Rationale for manual server-side configuration in the PoC

In a production setting, Zabbix templates and items can be deployed automatically via the Zabbix API. In this PoC, server-side configuration is intentionally kept manual for two reasons:

1. to avoid introducing administrative credentials and API write access into the deployment toolchain; and
2. to keep the proof-of-concept focused on *host-side telemetry enablement*, which was the principal missing capability.

4.7 Security and operational considerations

Although the design avoids installing new software, it still introduces a security-relevant control surface: the Zabbix agent executes external scripts. Therefore, the framework must ensure that scripts are:

- stored in a root-owned directory with controlled permissions,
- minimal and auditable, and
- robust to malformed output and unexpected tool behaviour.

Privilege is a practical concern. Some interfaces (e.g., certain sysfs energy counters or GPU telemetry paths) may not be readable by the unprivileged Zabbix agent user depending on system policy. The PoC assumes privileged deployment and emphasises fail-cleanly behaviour. A production hardening step would explicitly define the permission model (e.g., group-based read access to `powercap/hwmon` files, or tightly scoped `sudo` rules for specific commands), while preserving the script contract and key interface.

4.8 Limitations and future work

The current implementation demonstrates that a compatibility-first approach can expose power telemetry across diverse hosts with minimal operational disruption. However, several limitations remain:

1. **Redfish/IPMI power is not yet fully implemented.** The architecture includes an explicit hook for platform-level telemetry, but endpoint normalisation and authentication handling require a dedicated iteration.
2. **AMD power telemetry is heterogeneous.** Some AMD systems do not expose stable RAPL-like domains; `hwmon` fallbacks exist but require careful per-platform validation.
3. **Discovery breadth is intentionally conservative.** The PoC prioritises low overhead by avoiding exhaustive enumeration tools. A future iteration should incorporate richer host inventory tooling (e.g., `lspci`, `lshw`, `dmidecode`) at deployment time, parse results into a structured component model, and select the best available telemetry method for each detected component.
4. **Validation methodology is pending.** A complete evaluation should compare host-reported telemetry (RAPL/GPU counters) against platform-level BMC readings or facility-level metering to quantify bias across load regimes (cf. the blind-spot discussion in Section 2.2).

Despite these limitations, the PoC establishes a scalable integration path: power telemetry becomes a first-class metric in Zabbix without requiring invasive software installation, enabling subsequent optimisation work (identifying always-on idle systems, inefficient accelerator workloads, or candidates for consolidation) to proceed on a measurable basis.

Chapter 5

Component Efficiency and Optimisation

In order to analyse the overall energy efficiency of a server, we need to break it down into its singular components. If we can find specific components that are especially energy consuming and replace those, the total energy consumption can be reduced with little effort.

We have access to two different datasets of ETH servers. The first is an inventory list [7] which includes all existing systems and their specifics, e.g., the manufacturers, model, and the last inventory update.

The second is a Zabbix database [1], as described earlier in the report.

However, even though Zabbix collects a lot of different data, it does not include the data about the actual power usage of servers. As a consequence of the multitude of data, we will be focusing on the most relevant power consuming components, i.e. the Power Supply Unit (PSU) and Central Processing Unit (CPU).

The PSU determines the conversion efficiency from Alternating Current (AC) to Direct Current (DC). We chose to focus on the PSU because old ones typically waste between 10-20% of input power as heat. In contrast to this, modern PSUs only waste approximately <5% of power.

The CPU consumes about 35-60% of the total system power under load. Newer CPUs have better performance per Watt (perf/W), i.e. fewer servers are needed for the identical workload. Additionally, each server uses less power for the same performance if the perf/W of the CPU is higher.

We preprocessed the data through filtering out all the information of non-important components. In the following analysis we have included only the most prominent and used PSU and CPU models since these have the most impact on the energy efficiency of the servers.

5.1 PSU

Unfortunately, the inventory does not contain explicit information about the PSUs; therefore, the types need to be inferred from the server chassis models listed. We extracted all physical server models, de-duplicated them, counted per-model frequency, then mapped each chassis to its typical PSU configuration, idle power, and full-load power, based on manufacturer specifications or standard configurations for these systems.

The idle Power describes how much power a PSU consumes while it is not actively performing its main function, e.g. a desktop sitting at login. The lower the idle power, the more energy efficient the whole system is. However, the PSU is usually more inefficient at low loads. The energy consumption still declines with lower idle power because the whole system uses less energy and therefore the PSU is wasting less energy. It is measured in Watts (W).

The full load refers to the maximum constant power output a PSU can provide. A PSU reaches full load under, e.g. scientific computing workloads or GPU rendering. A high-quality PSU can run at full load continuously if it is 80 Plus certified. Nevertheless, a PSU is most efficient at 40-60% load. Running a PSU at full load for longer periods of time can lead to more energy loss through heat and increased fan function. The lower a PSU's full load, the more energy efficient it is. It is measured in Watts (W).

Table 5.1: Typical PSU capacities and power usage for systems in the inventory.

Physical Model	Server	Typical PSU	Idle Power (W)	Full Load (W)
Intel S2600 series[8]		2x 750 W Platinum hot-swap	85-120	250-400
GIGABYTE R-series (EPYC/Xeon)[9]		2x 1200 W / 1600 W Titanium	110-180	350-700
GIGABYTE G482 (GPU server)[10]		2x 2200 W Titanium	250-350	1200-2000
Intel M50CYP series[11]		2x 1300 W Platinum	70-110	220-350
Intel S4600LH[12]		2x 750 W Platinum	70-100	200-300
Desktops (OptiPlex / EliteDesk / ASUS)[13][14][15]		Internal ATX or proprietary PSUs (e.g., Dell OptiPlex 7060, ASUS B365M-K, HP EliteDesk 800 G3)	15-40	80-200

On the one hand, the most energy-efficient PSUs overall are desktop PSUs. This is to be expected, since they are usually utilised for smaller workloads. For a server-grade PSU, the Intel S4600LH has the most energy-efficient one, with a full-load on the lower end. On the other hand, the GIGABYTE G482 server requires two 2200 W Titanium PSUs. The idle power is as high as the full-load of the Intel S4600LH and the full-load goes up to 2000 W which is much higher than all of the other listed PSUs. However, this is also a GPU server which is utilised for high-performance and heavy-compute workloads.

5.1.1 Replacement options PSU

In the following segment, we provide some replacement options for the more energy efficient PSUs. This list was curated through extensive online research and comparing specifications provided by manufacturers. Nonetheless, we do not recommend replacing all of the existing hardware instantaneously. These are options to consider if and when updates are required.

Table 5.2: Typical replacement PSU capacities and power usage for systems in the inventory.

Replacement PSU	Typical Efficiency	Est. Idle Power (W)	Est. Full Load (W)
Supermicro PWS-1K22-1R 1200 W (redundant)[16]	$\approx 88-92\%$	$\approx 70-100$	$\approx 300-450$
Supermicro PWS-1K22A-1R 1200 W (Titanium)[17]	$\approx 90-94\%$	$\approx 65-95$	$\approx 280-430$
be quiet! Straight Power 12 1200 W[18]	up to 93.7% at 50% load	$\approx 50-80$	$\approx 250-400$
SilverStone SST-ST1200-PT 1200 W[19]	$\approx 90-94\%$	$\approx 55-85$	$\approx 260-420$

The first two PSUs are server-grade, while the second two are used for desktops. All of them generate less heat and therefore require less cooling, which leads to lower operating costs. With our replacement options, idle power drops by around 10-30 W on average.

5.2 CPU

Furthermore, we have explicit data regarding the CPU models and their usage frequency. In combination with additional research, e.g. through manufacturer data sheets, we are able to provide a concise analysis of CPU efficiency.

The idle power of a CPU is the energy it consumes while it is not actively used, e.g. while the system is running background tasks. The CPU enters a low-power state, i.e. C-state. The clock speed, i.e. the rate at which a CPU executes instructions, drops, and voltage is reduced. The idle power of a CPU is especially important, because a CPU spends most of its time idle. Therefore, the lower the idle power, the better the energy efficiency. It is measured in Watts (W).

The full-load power of a CPU is the actual electrical power it draws when fully utilised, e.g. during heavy computation. Energy consumption reaches its peak during full-load utilisation. It is measured in Watts (W).

The Thermal Design Power (TDP) is the thermal limit, defined by the CPU manufacturer, representing the maximum sustained heat the cooling system must be able to endure and

disperse under a standardised workload. It defines which CPUs a motherboard supports, the required cooling performance of a server, and the thermals and airflow needed in a server chassis. In general, a lower TDP is more energy efficient because less energy is required for cooling. However, a lower TDP does not automatically mean a CPU is more energy efficient. A modern high-TDP CPU can still be more efficient per unit of work compared to an older low-TDP CPU because of the servers overall architecture.

Table 5.3: Typical CPU capacities and power usage for systems in the inventory.

Model Group	TDP Range (W)	Idle System (W)	Full Load (W)
AMD EPYC 7002/7003 Servers (Rome)[20][21]	120-225	25-40	180-270
Intel Xeon S2600 Family[22]	205	20-40	150-220
Intel Xeon E5/ E5-4600 (legacy)[12]	85-155	10-25	90-150

The CPUs are ranked from least energy-efficient to most energy-efficient.

5.2.1 Replacement options CPU

In the following segment, we provide some replacement options for the more energy efficient CPUs. This list was curated through extensive online research and comparing specifications provided by manufacturers. Nevertheless, current CPUs should not be replaced while they still work well. These are simply options for the case that updates are required.

Table 5.4: Typical replacement CPU capacities and power usage for systems in the inventory.

Replacement CPU	Est. TDP Range (W)	Est. Idle System (W)	Est. Full Load (W)
Intel Xeon Silver/Gold 3rd Gen (Ice Lake)[23]	≈150-165	≈20-45	≈120-180
Intel Xeon 3rd Gen (3rd gen Scalable)[24]	up to 270	25-45	180-270
AMD EPYC 9004 (Genoa)[25]	200-400	30-60	200-400
Desktop: Intel Alder Lake/Raptor Lake (12th/13th gen)	35-125	5-20	40-150
Desktop: AMD Ryzen 5000/7000 series (Zen 3/4)[26]	65-105	5-20	50-150
Desktop Core i5/i7 (OptiPlex / EliteDesk / consumer boards)[27]	35-125	5-25	40-150

The listed CPUs have a better perf/W, can maintain their performance better under 24/7 load, and are able to consolidate workloads.

Chapter 6

Conclusion

In conclusion, the data on different server components are not fully complete. This impedes the analysis of power consumption. However, with the data provided, we can conclude that there can be significant improvements regarding energy efficiency.

For immediate improvements, we propose a few inexpensive and low-effort options. The first is to replace idle office desktops with a “Next Unit of Computing” (NUC)[28]. These are very small, ultra-compact computers originally designed by Intel. They have very low idle power (5–10 W), enough performance for office work, labs, thin clients, light servers and can replace many full desktop towers while using 80-90% less energy. Alternatively, a “Small Form Factor” (SFF) can be used. This is a compact desktop PC that is smaller than a full tower but larger than an NUC. The idle power is at 10-25 W. The advantage compared to an NUC is the possibility of effortlessly upgrading it. Another option to reduce power consumption immediately is to power-cap older servers where possible, e.g. IPMI/Redfish can set caps.

Regarding medium- and long term investments and improvements, we recommend replacing the highest energy consumers. These are three highest energy consumers that have the highest absolute power draw as a class and therefore the biggest potential savings if changed.

In first place are GIGABYTE G482 GPU Servers with two 2200 W PSUs. They have extremely large PSUs and are designed for GPU workloads, meaning power use can reach up to 2000 W. Additionally, they have a typically high idle draw, because of the overhead of GPU fans, baseboards, and redundant PSUs.

In second place are AMD EPYC 7002/7003 R-Series Servers. They have dual redundant 1200-1600 W Titanium PSUs and a high idle power. Since these systems run continuously, they represent a significant operational cost. Migration to the AMD EPYC 7003 Milan or 9004 Genoa machine can significantly reduce energy per unit of work.

The third place are Intel Xeon Scalable S2600-Series Servers. They have dual redundant 750-1100 W PSUs. Especially the older generations of Xeon Scalable are inefficient under virtualisation. The TDP per CPU can be up to 205 W. Considering that many of these are in the system, the impact is quite large. Migration to third-gen Xeon (Ice Lake) or AMD EPYC offers major perf/W gains.

In this report, we have contributed to the analysis of optimising computing energy of the D-ITET department. By analysing the current state of monitoring via Zabbix, we

identified significant gaps in the availability of power data. To address this, we proposed a multi-level measurement strategy utilising Facility, Server, System, and Component level sensors, and implemented a proof-of-concept power monitoring framework.

Our analysis highlights the critical importance of understanding load-dependent accuracy deviations, particularly the “Blind Spot” phenomenon in RAPL and GPU counters at idle states. The next phase of this project involves at-scale deployment of the discussed Bash scripts to gather unified data and the physical validation of these measurements via ampere meters. This will support accurate energy profiling and subsequent optimisation of the infrastructure.

Bibliography

- [1] Zabbix Official Website. <https://www.zabbix.com>. Accessed: 29 November 2025.
- [2] Aleksandar Radovanovic and et al. Power Modeling for Effective Datacenter Planning and Compute Management. *IEEE Transactions on Smart Grid*, 2021.
- [3] DSP2046 - Redfish Power Equipment Data Model Specification. https://www.dmtf.org/sites/default/files/standards/documents/DSP2046_2021.1.pdf. Accessed: 29 November 2025.
- [4] D. Hackenberg and et al. Power Measurement Techniques on Standard Compute Nodes: A Quantitative Comparison. 2015.
- [5] NVML API Reference - Power Profiles. https://docs.nvidia.com/deploy/nvml-api/group__nvmlPowerProfiles.html. Accessed: 29 November 2025.
- [6] ROCm system management interface library. https://github.com/ROCm/rocm_smi_lib. Accessed: 29 November 2025.
- [7] ETH Zürich — Invexseth Computer Overview. <https://invcxseth.ethz.ch/front/computer.php>. Accessed: 29 November 2025.
- [8] Intel Server Boards Overview (S2600WT / S2600WFT / S2600GZ / S4600LH). Intel Server Boards and Chassis Technical Product Specifications. Accessed: 29 November 2025.
- [9] GIGABYTE / Giga Computing R-Series Servers. <https://www.gigabyte.com/Enterprise>. Accessed: 29 November 2025.
- [10] GIGABYTE G482-Z54 GPU Server. <https://www.gigabyte.com/High-Performance-Computing/G482-Z54-rev-100>. Accessed: 29 November 2025.
- [11] Intel M50CYP2S / M50CYP Server Family. <https://www.intel.com/content/www/us/en/products/docs/servers/server-systems/m50cyp-family.html>. Accessed: 29 November 2025.
- [12] Intel Xeon E5-4600 Processor Family — ARK. <https://ark.intel.com/content/www/us/en/ark/products/series/59139/intel-xeon-processor-e54600-product-family.html>. Accessed: 29 November 2025.

- [13] Dell OptiPlex 7060 — Technical Guidebook. Dell OEM Technical Documentation. Accessed: 29 November 2025.
- [14] HP EliteDesk 800 G3 — HP QuickSpecs. HP OEM Documentation. Accessed: 29 November 2025.
- [15] ASUS / MSI Consumer Motherboards — ATX12V PSU Specification. ATX PSU Standard (450–650 W Typical). Accessed: 29 November 2025.
- [16] Supermicro PWS-1K22-1R 1200 W 1U Redundant Power Supply. <https://www.networktigers.com/products/pws-1k22-1r-supermicro-power-supply>. Accessed: 29 November 2025.
- [17] Supermicro PWS-1K22A-1R 1200 W 1U Redundant Power Supply (Titanium). <https://www.pbtech.com/au/product/PSUSPM12201/Supermicro-PWS-1K22A-1R-1200W-1U-Redundant-Power-S>. Accessed: 29 November 2025.
- [18] be quiet! Straight Power 12 1200 W ATX Power Supply. <https://www.bequiet.com/en/powersupply/4105>. Accessed: 29 November 2025.
- [19] SilverStone SST-ST1200-PT / ST1200-PT 1200 W 80 PLUS Platinum Power Supply. <https://www.silverstonetek.com/en/product/info/power-supplies/ST1200-PT/>. Accessed: 29 November 2025.
- [20] AMD EPYC 7002 Series. <https://www.amd.com/en/products/server-processors/epyc-7002-series>. Accessed: 29 November 2025.
- [21] AMD EPYC 7003 Series. <https://www.amd.com/en/products/server-processors/epyc-7003-series>. Accessed: 29 November 2025.
- [22] Intel Server Board S2600WF / WFT / WT / GZ Technical Product Specifications. Specific TPS PDFs from Intel Download Center. Accessed: 29 November 2025.
- [23] 3rd Gen Intel Xeon Silver and Gold Processors (Ice Lake). <https://ark.intel.com/content/www/us/en/ark/products/series/204098/3rd-gen-intel-xeon-scalable-processors.html>. Accessed: 29 November 2025.
- [24] 3rd Gen Intel Xeon Scalable Processors — TDP Specifications. <https://www.intel.com/content/www/us/en/products/details/processors/xeon/scalable/3rd-gen.html>. Accessed: 29 November 2025.
- [25] AMD EPYC 9004 Series (Genoa) Server Processors. <https://www.amd.com/en/products/server-processors/epyc-9004-series>. Accessed: 29 November 2025.
- [26] AMD Ryzen 5000 and 7000 Series Desktop Processors (Zen 3 / Zen 4). <https://www.amd.com/en/products/processors/desktops>. Accessed: 29 November 2025.
- [27] Intel Core i5/i7 Desktop Processors — ARK Specifications. <https://ark.intel.com/>. Accessed: 29 November 2025.
- [28] Intel NUC Systems — Product Pages. <https://www.intel.com/content/www/us/en/products/details/nuc.html>. Accessed: 29 November 2025.